

```
In [1]: import pandas as pd
```

This loads the CVS file into a Pandas DataFrame df

```
In [2]: df = pd.read_csv("trends.csv")
df
```

```
Out[2]:
```

	location	year	category	rank	query
0	Global	2001	Consumer Brands	1	Nokia
1	Global	2001	Consumer Brands	2	Sony
2	Global	2001	Consumer Brands	3	BMW
3	Global	2001	Consumer Brands	4	Palm
4	Global	2001	Consumer Brands	5	Adobe
...	...	...	...	...	...
26950	Vietnam	2020	Là Gì?	1	Virus Corona là gì
26951	Vietnam	2020	Là Gì?	2	Miễn thị thực là gì
26952	Vietnam	2020	Là Gì?	3	Đầu cắt mồi là gì
26953	Vietnam	2020	Là Gì?	4	Bệnh bạch hầu là gì
26954	Vietnam	2020	Là Gì?	5	Đông Lào là gì

26955 rows × 5 columns

The head() function displays the first five rows of the dataset.

```
In [3]: df.head()
```

```
Out[3]:
```

	location	year	category	rank	query
0	Global	2001	Consumer Brands	1	Nokia
1	Global	2001	Consumer Brands	2	Sony
2	Global	2001	Consumer Brands	3	BMW
3	Global	2001	Consumer Brands	4	Palm
4	Global	2001	Consumer Brands	5	Adobe

The tail() function displays the last five rows of the dataset.

```
In [4]: df.tail()
```

Out[4]:

	location	year	category	rank	query
26950	Vietnam	2020	Là Gì?	1	Virus Corona là gì
26951	Vietnam	2020	Là Gì?	2	Miến thị thực là gì
26952	Vietnam	2020	Là Gì?	3	Đầu cắt moi là gì
26953	Vietnam	2020	Là Gì?	4	Bệnh bạch hầu là gì
26954	Vietnam	2020	Là Gì?	5	Đông Lào là gì

- **shape displays the number of rows and columns.**
- **The dataset contains 26,955 rows and 5 columns.**

In [5]: `df.shape`

Out[5]: (26955, 5)

- **info() function displays the complete information about the dataset.**
- **The dataset contains integer columns(year and rank)**
- **object/text columns(location, category, and query).**

In [6]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26955 entries, 0 to 26954
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   location    26955 non-null  object
1   year        26955 non-null  int64
2   category    26955 non-null  object
3   rank        26955 non-null  int64
4   query       26955 non-null  object
dtypes: int64(2), object(3)
memory usage: 1.0+ MB
```

- **The describe() function provides statistical information about numerical columns.**
- **including count, mean, standard deviation, minimum, maximum, and quartile values.**

In [7]: `df.describe()`

Out[7]:

	year	rank
count	26955.000000	26955.000000
mean	2015.243369	3.000000
std	3.564683	1.41424
min	2001.000000	1.000000
25%	2013.000000	2.000000
50%	2016.000000	3.000000
75%	2018.000000	4.000000
max	2020.000000	5.000000

- **Displays column names.**
- **The dataset contains five columns: location, year, category, rank, and query.**

In [8]: `df.columns`

Out[8]: Index(['location', 'year', 'category', 'rank', 'query'], dtype='object')

- **The dtypes function shows the data type of each column.**
- **(year and rank)integers.**
- **(location, category, and query) object/text values.**

In [9]: `df.dtypes`

```
Out[9]: location    object
        year       int64
        category   object
        rank       int64
        query      object
        dtype: object
```

- **The isnull().sum() function counts missing values in each column.**
- **There are no missing values in the dataset.**

In [10]: `df.isnull().sum()`

```
Out[10]: location    0
        year       0
        category    0
        rank       0
        query      0
        dtype: int64
```

- **The duplicated().sum() function counts the number on duplicates records.**
- **The dataset contains 10 completely duplicated records.**

In [11]: `df.duplicated().sum()`

```
Out[11]: np.int64(10)
```

- **The nunique() function shows the number of distinct values in each column.**
- **The dataset has 83 locations, 20 years, 2,450 categories, 5 ranks, and 18,431 unique queries.**

```
In [12]: df.nunique()
```

```
Out[12]: location      83
         year         20
         category    2450
         rank         5
         query     18431
         dtype: int64
```

- **The mean represents the average value of the numerical columns.**
- **this give the average value of the columns(year, rank).**

```
In [13]: df[["year", "rank"]].mean()
```

```
Out[13]: year    2015.243369
         rank      3.000000
         dtype: float64
```

- **The median represents the middle value when the observations are arranged in order.**
- **This give the the middle value of the columns(year,rank).**

```
In [14]: df[["year", "rank"]].median()
```

```
Out[14]: year    2016.0
         rank      3.0
         dtype: float64
```

- **The min() function identifies the smallest value in each numerical column.**
- **This gives the smallest value of the columns(year, rank).**

```
In [15]: df[["year", "rank"]].min()
```

```
Out[15]: year    2001
         rank      1
         dtype: int64
```

- **The max() function identifies the largest value.**
- **This gives the largest value of the columns(year, rank).**

```
In [16]: df[["year", "rank"]].max()
```

```
Out[16]: year    2020
         rank      5
         dtype: int64
```

- **Standard deviation measures how much the numerical values vary around their mean.**
- **A larger value indicates greater variation.**

```
In [17]: df[["year", "rank"]].std()
```

```
Out[17]: year      3.564683  
rank       1.414240  
dtype: float64
```

- **The year values range from 2001 to 2020 and rank values range from 1 to 5.**
- **These values appear reasonable for the dataset, and no obvious invalid numerical values are observed.**

```
In [18]: print("Year range:", df["year"].min(), "to", df["year"].max())  
print("Rank range:", df["rank"].min(), "to", df["rank"].max())
```

```
Year range: 2001 to 2020  
Rank range: 1 to 5
```

```
In [19]: df["rank"].unique()
```

```
Out[19]: array([1, 2, 3, 4, 5])
```

Overall interpretation

- The dataset contains 26,955 rows and 5 columns: location, year, category, rank, and query.
- The dataset has no missing values, but 10 duplicate records were identified and should be reviewed before further analysis.
- The data types are appropriate for the respective columns.
- There are 83 unique locations, 20 unique years, 2,450 categories, 5 rank values, and 18,431 unique queries.
- The numerical values appear reasonable, with years ranging from 2001 to 2020 and ranks ranging from 1 to 5.
- Further cleaning may focus on duplicate records and checking text columns for inconsistent formatting.

```
In [20]: df.drop_duplicates(inplace=True)
```

```
In [22]: df.duplicated().sum()
```

```
Out[22]: np.int64(0)
```

```
In [ ]:
```